Tecnológico de Monterrey

Maestría: Inteligencia artificial aplicada (MNA)

Materia:Visión computacional para imágenes y video

Profesor Titular: PhD. Gilberto Ochoa Ruiz.

Actividad: E-4 Google Colab - Extracción de líneas con algoritmo de Canny

Integrantes:

- A01754247 - Dannae Abed Ramírez Cohassarco
- A01754262 - Yoselly Hernández Martínez
- A01754268 - Ana Sofía Ramírez Ramírez
- A01754279 - Octavio Rosas Ramírez Guerrero
- A01754282 - José Fabian López Cruz

Instrucciones:

Lo realizado en este de reporte y deberá ser contestar en los frames deberá:

Comprobar que los ejemplos de Canny funcionan con la ejecución de celda.

Comprobarlo con imágenes propias y documentar en el reporte de Google Colab cómo se comporta el procedimiento del algoritmo.

Detector Canny (Teoría)

La detección de bordes es una de las técnicas más utilizadas para los permite saber los objetos y características del frame. El detector de bordes de Canny es un operador de detección de bordes que utiliza un algoritmo de varias etapas para detectar una amplia gama de bordes en imágenes. Fue ideado por John F. Canny. Es considerado el borde y es creado John F. Canny que lo desarrolla en 1986 como un algoritmo de detección de bordes óptimo. Su objetivo es cumplir con tres criterios de detección y encontrar una solución para ellos.

Algunos de los criterios de detección que considera el algoritmo de Canny son:

- Relación de Ruido mediante suavizado Gaussiano
- Localización de los bordes mediante el gradiente
- Supresión No Máxima para obtener bordes finos
- Aplicación del doble umbral
- Seguimiento de bordes por histéresis

Para más información ahondar en imágenes con ejemplo de ejemplo.

1 Reducción de Ruido:

El ruido es muy común debido a que los ultimos mantenimientos en la detección de bordes están basados en derivadas, por lo que sin ellas al obtendríaque ganancias para suavizar la imagen. Para ello, se aplica la técnica de convolución de imágenes con el filtro Gaussiano (5X5, 5X7, 7X7, etc.). Cuanto más pequeño sea el kernel, menos borroso será el suavizado.

2 Cálculo del Gradiente:

Aquí se detecta la intensidad y dirección de los bordes calculando el gradiente de la imagen mediante operadores de detección de bordes. Los bordes corresponden a un cambio de brusco de intensidad de los pixeles. Para detectar los bordes de una imagen, hay varios operadores que pueden ser aplicados (Prewitt, Sobel, Roberts y Canny).

3 Supresión No Máxima:

Se aplica a todos los pixeles del gradiente. Es la mejor manera para obtener bordes finos, sin embargo, once que nos grasas pixeles, y a pesarlos filtro aquella a eliminarlos los bordes mas posibles.

El algoritmo recorre todos los puntos de la matriz de intensidad del gradiente y comprueba los pixeles con el valor máximo en las direcciones de los bordes. Tal vez a ocasión para ver si el pixel actual es el pixel con el valor más alto en su vecindad en la dirección del gradiente. Si es así, el pixel pasa a la siguiente etapa, si no es así, el pixel cuyo valor no es el máximo, se establece en cero. Como resultado, se obtiene una imagen binaria de bordes finos.

4 Doble Umbral de Histéresis:

En este paso se aplica el doble umbral a los pixeles que quedan. Si los pixeles que quedan tienen un valor por encima del umbral superior, se establecen como bordes fuertes. Si los pixeles que quedan tienen un valor por debajo del umbral inferior, se establecen como no bordes. Si los pixeles que quedan tienen un valor entre los dos umbrales, el pixel pasa a la siguiente etapa, si no es así, se establece como no borde.

5 Seguimiento de Bordes por Histéresis:

Finalmente se analizan los pixeles que quedan. Los bordes fuertes son aceptados, pero aquellos que identificaron los pixeles no relacionados (intensidad bajo) al umbral bajo. Todos los pixeles que corresponden se encuentra entre ambos umbrales se marcan como débiles.

6 Rastreo de bordes por histéresis:

Por ultimo, si un pixel débil tiene como vecino a un pixel fuerte, se transforma a pixel fuerte.

No strong pixels around

One strong pixel around

2. Sobel and Canny Edge Detection

Table of Contents

- Libraries
- Sobel Edge Detection
- Canny Edge Detection

Importing Libraries

```
# Numpy for array manipulation
import numpy as np
# Matplotlib for plotting
import matplotlib
from matplotlib import pyplot
# Matplotlib inline to show plots in notebook
%matplotlib inline
# Matplotlib pyplot for easier plotting
from matplotlib.pyplot import plt
# Cv2 for computer vision
import cv2
# Math for mathematical functions
import math
# Time for measuring execution time
import time
# Os for interacting with the operating system
import os
# Glob for finding files matching a pattern
import glob
# Pandas for data manipulation and analysis
import pandas as pd
# Seaborn for statistical data visualization
import seaborn as sns
# Sklearn for machine learning
import sklearn
# Keras for deep learning
import keras
# Tensorflow for deep learning
import tensorflow as tf
# Torch for deep learning
import torch
# Torchvision for computer vision
import torchvision
```

```
[nltk_data] Downloading stopwords.zip.
[nltk_data] Unzipping corpora/stopwords.zip.
```

```
%matplotlib inline
plt = lambda x=None: print("Using matplotlib") if x is None else print("Using matplotlib", x)
```

```
Using matplotlib
```

```
# Read the image.
img = cv2.imread('images/Gradient_Image.png', 0)
plt.imshow(img, cmap = 'gray')
plt.title('Original Gradient Image')
plt.xticks([]), plt.yticks([])
```

```
# Gaussian Noise.
noise = np.zeros(img.shape, dtype=np.uint8)
cv2.randn(noise, 0, 25)
noisy = img + noise
plt.imshow(noise, cmap = 'gray')
plt.title('Gaussian Noise')
plt.xticks([]), plt.yticks([])
```

```
# Add noise to the image.
noisy = img + noise
plt.imshow(noisy, cmap = 'gray')
plt.title('Gaussian Noise Added')
plt.xticks([]), plt.yticks([])
```

Now we'll try with the more blurred gaussian filtered image.

```
# Read the image.
img = cv2.imread('images/Blurred_Image.png', 0)
plt.imshow(img, cmap = 'gray')
plt.title('Blurred Gradient Image')
plt.xticks([]), plt.yticks([])
```

```
# Gaussian Noise.
noise = np.zeros(img.shape, dtype=np.uint8)
cv2.randn(noise, 0, 25)
noisy = img + noise
plt.imshow(noise, cmap = 'gray')
plt.title('Gaussian Noise')
plt.xticks([]), plt.yticks([])
```

```
# Add noise to the image.
noisy = img + noise
plt.imshow(noisy, cmap = 'gray')
plt.title('Gaussian Noise Added')
plt.xticks([]), plt.yticks([])
```

Sobel Edge Detection

Sobel is also known as emphasizing features, you will apply the Sobel edge detection algorithm. This finds regions of the image with large gradient values in multiple directions. Regions with high constitutional gradient are likely to be edges or transitions in the pixel values.

The code is the off below applies the Sobel algorithm to the marble Filtered image, using these steps:

1. Convert the image to grayscale, if it is not already in grayscale (also assume it is two dimensional)
2. Compute the gradient in the x direction using the Sobel operator, and the absolute value of the gradient.
3. Compute the magnitude of the gradient.
4. Normalize the gradient matrix.

```
# Read the image.
img = cv2.imread('images/Marble_Filtered_Image.jpg', 0)
# Calculate the Sobel gradient.
sobelx = cv2.Sobel(img,cv2.CV_64F,1,0,ksize=5)
sobelx = np.absolute(sobelx)
# Normalize the gradient.
sobelx = 255*(sobelx/np.max(sobelx))
# Plot the original image.
plt.imshow(img, cmap = 'gray')
plt.title('Original Image')
plt.xticks([]), plt.yticks([])
```

```
# Plot the Sobel gradient.
plt.imshow(sobelx, cmap = 'gray')
plt.title('Sobel Gradient')
plt.xticks([]), plt.yticks([])
```

Canny Edge Detection

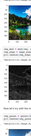
Steps:

1. Noise Reduction
2. Gradient Calculation
3. Non-maximum Suppression
4. Double Threshold
5. Edge Tracking by Hysteresis

Pre-requisite: Convert the image to grayscale before algorithm.

```
# Read the image.
img = cv2.imread('images/Lena.jpg', 0)
# Canny edge detection.
img_canny = cv2.Canny(img,100,200)
# Plot original image.
plt.imshow(img, cmap = 'gray')
plt.title('Original Image')
plt.xticks([]), plt.yticks([])
```

```
# Plot canny edge detected image.
plt.imshow(img_canny, cmap = 'gray')
plt.title('Canny Edge Detected Image')
plt.xticks([]), plt.yticks([])
```



1.Noise Reduction

Edge detection is very sensitive to image noise so the derivatives behind the algorithm.

We will apply a Gaussian filtrate, the size of the kernel dependes on the experienced blurring effect. The smaller the less blurring effect.

Equation for Gaussian kernel of size (2k + 1) × (2k + 1):

$$H_{ij} = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{(i-(k+1))^2 + (j-(k+1))^2}{2\sigma^2}\right); 1 \leq i, j \leq (2k+1)$$

```
# Read the image.
img = cv2.imread('images/Lena.jpg', 0)
# Plot the original image.
plt.imshow(img, cmap = 'gray')
plt.title('Original Image')
plt.xticks([]), plt.yticks([])
```

```
# Gaussian Blur.
img = cv2.GaussianBlur(img, (5, 5), 0)
plt.imshow(img, cmap = 'gray')
plt.title('Gaussian Blurred Image')
plt.xticks([]), plt.yticks([])
```

1.2 Signal Size Parameter

```
Using pyplot
```

```
# Plot the original image.
plt.imshow(img, cmap = 'gray')
plt.title('Original Image')
plt.xticks([]), plt.yticks([])
```

1.3 Signa Parameter .

```
# Plot the blurred image.
plt.imshow(img, cmap = 'gray')
plt.title('Blurred Image')
plt.xticks([]), plt.yticks([])
```

2. Gradient Calculation

Edges correspond to a change of pixel intensity.

To detect it, the easiest way is to apply filters that highlight this intensity change in both directions:

- horizontal (x)
- vertical (y)

A pixel is characterized by a intensity I with Sobel kernels K_x and K_y

$$K_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix} \quad K_y = \begin{bmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

Then, the magnitude G and the slope Θ of the gradient are calculated as follow:

$$G = \sqrt{G_x^2 + G_y^2} \quad \Theta = \arctan\left(\frac{G_y}{G_x}\right)$$

```
# Read the image.
img = cv2.imread('images/Lena.jpg', 0)
# Gaussian Blur.
img = cv2.GaussianBlur(img, (5, 5), 0)
# Calculate the x and y gradients.
gx = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=5)
gy = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=5)
# Calculate the magnitude.
g = np.sqrt(gx**2 + gy**2)
# Normalize the magnitude.
g = 255*(g/np.max(g))
# Plot the gradient magnitude.
plt.imshow(g, cmap = 'gray')
plt.title('Gradient Magnitude')
plt.xticks([]), plt.yticks([])
```

3. Non-Maximum suppression

- Create a matrix initialized to 0 of the same size of the original gradient intensity matrix.
- Identify the neighborhood of each pixel in the matrix of gradient intensities.
- Compare the angle of the same neighborhood has a higher intensity than the pixel that is currently processed.
- Return the image processed with the non-max suppression algorithm.

```
# Calculate the angle.
a = np.arctan2(gy, gx)
# Plot the angle.
plt.imshow(a, cmap = 'gray')
plt.title('Angle')
plt.xticks([]), plt.yticks([])
```



```
# Get the dimensions of the image.
n,m = g.shape
# Set the angle to 0 if it is negative.
a = a.clip(0.0,0.0)
# Loop through the image.
for i in range(1,n-1):
    for j in range(1,m-1):
        # Get the angle.
        q = a[i,j]
        # If the angle is 0, 45, 90, or 135 then compare to the pixels in the gradient.
        if q == 0:
            q = max([g[i,j-1], g[i,j], g[i,j+1]])
        elif q == 45:
            q = max([g[i-1,j+1], g[i,j], g[i+1,j-1]])
        elif q == 90:
            q = max([g[i-1,j], g[i,j], g[i+1,j]])
        else:
            q = max([g[i-1,j-1], g[i,j], g[i+1,j+1]])
        # If the pixel is not the maximum, set it to 0.
        if g[i,j] != q:
            g[i,j] = 0
# Plot the image.
plt.imshow(g, cmap = 'gray')
plt.title('Non-Max Suppression')
plt.xticks([]), plt.yticks([])
```


4. Double threshold

- Strong pixels are pixels that have an intensity so high that we are sure they contribute to the final edge.
- Weak pixels are pixels that have an intensity value that is not enough to be considered as strong ones, but yet small enough to be considered as non-relevant for the edge detection.
- Other pixels are considered as non-relevant for the edge.

```
# Set the thresholds.
minVal = np.mean(g) * 0.1
maxVal = np.mean(g) * 0.3
# Get the dimensions of the image.
n,m = g.shape
# Define the array of zeros.
strong = np.zeros(g.shape)
weak = np.zeros(g.shape)
# Loop through the image.
for i in range(1,n-1):
    for j in range(1,m-1):
        # If the pixel is a strong pixel.
        if g[i,j] > maxVal:
            strong[i,j] = 255
        # If the pixel is a weak pixel.
        elif g[i,j] > minVal:
            weak[i,j] = 255
# Plot the strong pixels.
plt.imshow(strong, cmap = 'gray')
plt.title('Strong Pixels')
plt.xticks([]), plt.yticks([])
```



```
# Plot the weak pixels.
plt.imshow(weak, cmap = 'gray')
plt.title('Weak Pixels')
plt.xticks([]), plt.yticks([])
```


5. Edge Tracking by Hysteresis

This heuristics consists of considering weak pixels into strong ones. If and only if at least one of the pixels around the one being processed is a strong one.

```
# Set the strong and weak pixels.
for i in range(1,n-1):
    for j in range(1,m-1):
        if strong[i,j] == 255:
            if weak[i,j] == 255:
                strong[i,j] = 255
                if weak[i-1,j-1] == 255 or weak[i,j-1] == 255 or weak[i,j+1] == 255 or weak[i+1,j-1] == 255 or weak[i-1,j] == 255 or weak[i+1,j] == 255 or weak[i-1,j+1] == 255 or weak[i,j+1] == 255 or weak[i+1,j+1] == 255:
                    weak[i,j] = 255
# Plot the strong pixels.
plt.imshow(strong, cmap = 'gray')
plt.title('Strong Pixels')
plt.xticks([]), plt.yticks([])
```



```
# Plot the weak pixels.
plt.imshow(weak, cmap = 'gray')
plt.title('Weak Pixels')
plt.xticks([]), plt.yticks([])
```


ACTIVIDAD PARTE A:

Lo realizado en este de reporte y deberá ser contestar en los frames deberá:

El cuadro en color artificial de celdas tomo las características bioquímicas en un otro de una imagen que extraíamos su artículo de la revista El Sitio Especialista en un artículo para concept. Comprobar que los ejemplos de Canny funcionan con la ejecución de celda. Comprobarlo con imágenes propias y documentar en el reporte de Google Colab cómo se comporta el procedimiento del algoritmo. Cada uno de los pixeles que forman una imagen digital contienen una cantidad de luz que es recibida por un sensor. Cada pixel tiene un valor numérico que representa la cantidad de luz que recibe el punto de la imagen que representa. Los rayos de luz, antes de ser recibidos por el sensor, pasan por un filtro de color rojo, verde o azul. Los sensores de la cámara digital reciben la información de los rayos de luz de cada color y almacenan la información de cada pixel.

De hay, una forma para determinar cual color de rayos es el correcto. El color está cuidado depende de cada bulgar y sus factores.

```
# Read the image.
img = cv2.imread('images/Cell_Image.jpg', 0)
# Plot the original image.
plt.imshow(img, cmap = 'gray')
plt.title('Original Cell Image')
plt.xticks([]), plt.yticks([])
```



```
# Gaussian Blur.
img = cv2.GaussianBlur(img, (5, 5), 0)
# Calculate the x and y gradients.
gx = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=5)
gy = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=5)
# Calculate the magnitude.
g = np.sqrt(gx**2 + gy**2)
# Normalize the magnitude.
g = 255*(g/np.max(g))
# Plot the gradient magnitude.
plt.imshow(g, cmap = 'gray')
plt.title('Gradient Magnitude')
plt.xticks([]), plt.yticks([])
```



```
# Calculate the angle.
a = np.arctan2(gy, gx)
# Plot the angle.
plt.imshow(a, cmap = 'gray')
plt.title('Angle')
plt.xticks([]), plt.yticks([])
```



```
# Get the dimensions of the image.
n,m = g.shape
# Set the angle to 0 if it is negative.
a = a.clip(0.0,0.0)
# Loop through the image.
for i in range(1,n-1):
    for j in range(1,m-1):
        # Get the angle.
        q = a[i,j]
        # If the angle is 0, 45, 90, or 135 then compare to the pixels in the gradient.
        if q == 0:
            q = max([g[i,j-1], g[i,j], g[i,j+1]])
        elif q == 45:
            q = max([g[i-1,j+1], g[i,j], g[i+1,j-1]])
        elif q == 90:
            q = max([g[i-1,j], g[i,j], g[i+1,j]])
        else:
            q = max([g[i-1,j-1], g[i,j], g[i+1,j+1]])
        # If the pixel is not the maximum, set it to 0.
        if g[i,j] != q:
            g[i,j] = 0
# Plot the image.
plt.imshow(g, cmap = 'gray')
plt.title('Non-Max Suppression')
plt.xticks([]), plt.yticks([])
```



```
# Set the thresholds.
minVal = np.mean(g) * 0.1
maxVal = np.mean(g) * 0.3
# Get the dimensions of the image.
n,m = g.shape
# Define the array of zeros.
strong = np.zeros(g.shape)
weak = np.zeros(g.shape)
# Loop through the image.
for i in range(1,n-1):
    for j in range(1,m-1):
        # If the pixel is a strong pixel.
        if g[i,j] > maxVal:
            strong[i,j] = 255
        # If the pixel is a weak pixel.
        elif g[i,j] > minVal:
            weak[i,j] = 255
# Plot the strong pixels.
plt.imshow(strong, cmap = 'gray')
plt.title('Strong Pixels')
plt.xticks([]), plt.yticks([])
```



```
# Plot the weak pixels.
plt.imshow(weak, cmap = 'gray')
plt.title('Weak Pixels')
plt.xticks([]), plt.yticks([])
```



```
# Set the strong and weak pixels.
for i in range(1,n-1):
    for j in range(1,m-1):
        if strong[i,j] == 255:
            if weak[i,j] == 255:
                strong[i,j] = 255
                if weak[i-1,j-1] == 255 or weak[i,j-1] == 255 or weak[i,j+1] == 255 or weak[i+1,j-1] == 255 or weak[i-1,j] == 255 or weak[i+1,j] == 255 or weak[i-1,j+1] == 255 or weak[i,j+1] == 255 or weak[i+1,j+1] == 255:
                    weak[i,j] = 255
# Plot the strong pixels.
plt.imshow(strong, cmap = 'gray')
plt.title('Strong Pixels')
plt.xticks([]), plt.yticks([])
```



```
# Plot the weak pixels.
plt.imshow(weak, cmap = 'gray')
plt.title('Weak Pixels')
plt.xticks([]), plt.yticks([])
```


ACTIVIDAD PARTE B:

Lo realizado en este de reporte y deberá ser contestar en los frames deberá:

El detector Canny es la norma como etapa de detección de bordes y también es el utilizado del problema. Si hagan muchísimo pulidos o ser tan sensacional la detección de los contornos, es decir, la aplicación de los filtros necesarios en la imagen. Con un rango de 50, todos los pixeles contienen a 0 y mayor ser cada complementarlo mayor.

Actividad Desarrollada

- Reducción de Ruido
- Gradiente de Intensidad
- Edge Tracking by Hysteresis

```
# Read the image.
img = cv2.imread('images/Cell_Image.jpg', 0)
# Plot the original image.
plt.imshow(img, cmap = 'gray')
plt.title('Original Cell Image')
plt.xticks([]), plt.yticks([])
```



```
# Gaussian Blur.
img = cv2.GaussianBlur(img, (5, 5), 0)
# Calculate the x and y gradients.
gx = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=5)
gy = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=5)
# Calculate the magnitude.
g = np.sqrt(gx**2 + gy**2)
# Normalize the magnitude.
g = 255*(g/np.max(g))
# Plot the gradient magnitude.
plt.imshow(g, cmap = 'gray')
plt.title('Gradient Magnitude')
plt.xticks([]), plt.yticks([])
```



```
# Calculate the angle.
a = np.arctan2(gy, gx)
# Plot the angle.
plt.imshow(a, cmap = 'gray')
plt.title('Angle')
plt.xticks([]), plt.yticks([])
```



```
# Get the dimensions of the image.
n,m = g.shape
# Set the angle to 0 if it is negative.
a = a.clip(0.0,0.0)
# Loop through the image.
for i in range(1,n-1):
    for j in range(1,m-1):
        # Get the angle.
        q = a[i,j]
        # If the angle is 0, 45, 90, or 135 then compare to the pixels in the gradient.
        if q == 0:
            q = max([g[i,j-1], g[i,j], g[i,j+1]])
        elif q == 45:
            q = max([g[i-1,j+1], g[i,j], g[i+1,j-1]])
        elif q == 90:
            q = max([g[i-1,j], g[i,j], g[i+1,j]])
        else:
            q = max([g[i-1,j-1], g[i,j], g[i+1,j+1]])
        # If the pixel is not the maximum, set it to 0.
        if g[i,j] != q:
            g[i,j] = 0
# Plot the image.
plt.imshow(g, cmap = 'gray')
plt.title('Non-Max Suppression')
plt.xticks([]), plt.yticks([])
```



```
# Set the thresholds.
minVal = np.mean(g) * 0.1
maxVal = np.mean(g) * 0.3
# Get the dimensions of the image.
n,m = g.shape
# Define the array of zeros.
strong = np.zeros(g.shape)
weak = np.zeros(g.shape)
# Loop through the image.
for i in range(1,n-1):
    for j in range(1,m-1):
        # If the pixel is a strong pixel.
        if g[i,j] > maxVal:
            strong[i,j] = 255
        # If the pixel is a weak pixel.
        elif g[i,j] > minVal:
            weak[i,j] = 255
# Plot the strong pixels.
plt.imshow(strong, cmap = 'gray')
plt.title('Strong Pixels')
plt.xticks([]), plt.yticks([])
```



```
# Plot the weak pixels.
plt.imshow(weak, cmap = 'gray')
plt.title('Weak Pixels')
plt.xticks([]), plt.yticks([])
```



```
# Set the strong and weak pixels.
for i in range(1,n-1):
    for j in range(1,m-1):
        if strong[i,j] == 255:
            if weak[i,j] == 255:
                strong[i,j] = 255
                if weak[i-1,j-1] == 255 or weak[i,j-1] == 255 or weak[i,j+1] == 255 or weak[i+1,j-1] == 255 or weak[i-1,j] == 255 or weak[i+1,j] == 255 or weak[i-1,j+1] == 255 or weak[i,j+1] == 255 or weak[i+1,j+1] == 255:
                    weak[i,j] = 255
# Plot the strong pixels.
plt.imshow(strong, cmap = 'gray')
plt.title('Strong Pixels')
plt.xticks([]), plt.yticks([])
```



```
# Plot the weak pixels.
plt.imshow(weak, cmap = 'gray')
plt.title('Weak Pixels')
plt.xticks([]), plt.yticks([])
```



```
# Plot the original image.
plt.imshow(img, cmap = 'gray')
plt.title('Original Image')
plt.xticks([]), plt.yticks([])
```

```
# Plot the strong pixels.
plt.imshow(strong, cmap = 'gray')
plt.title('Strong Pixels')
plt.xticks([]), plt.yticks([])
```



```
# Plot the weak pixels.
plt.imshow(weak, cmap = 'gray')
plt.title('Weak Pixels')
plt.xticks([]), plt.yticks([])
```



```
# Plot the gradient magnitude.
plt.imshow(g, cmap = 'gray')
plt.title('Gradient Magnitude')
plt.xticks([]), plt.yticks([])
```



```
# Plot the angle.
plt.imshow(a, cmap = 'gray')
plt.title('Angle')
plt.xticks([]), plt.yticks([])
```



```
# Plot the non-max suppression.
plt.imshow(g, cmap = 'gray')
plt.title('Non-Max Suppression')
plt.xticks([]), plt.yticks([])
```



```
# Plot the double threshold.
plt.imshow(strong, cmap = 'gray')
plt.title('Double Threshold')
plt.xticks([]), plt.yticks([])
```



```
# Plot the edge tracking by hysteresis.
plt.imshow(strong, cmap = 'gray')
plt.title('Edge Tracking by Hysteresis')
plt.xticks([]), plt.yticks([])
```



```
# Plot the original image.
plt.imshow(img, cmap = 'gray')
plt.title('Original Image')
plt.xticks([]), plt.yticks([])
```



```
# Plot the gradient magnitude.
plt.imshow(g, cmap = 'gray')
plt.title('Gradient Magnitude')
plt.xticks([]), plt.yticks([])
```



```
# Plot the angle.
plt.imshow(a, cmap = 'gray')
plt.title('Angle')
plt.xticks([]), plt.yticks([])
```



```
# Plot the non-max suppression.
plt.imshow(g, cmap = 'gray')
plt.title('Non-Max Suppression')
plt.xticks([]), plt.yticks([])
```



```
# Plot the double threshold.
plt.imshow(strong, cmap = 'gray')
plt.title('Double Threshold')
plt.xticks([]), plt.yticks([])
```



```
# Plot the edge tracking by hysteresis.
plt.imshow(strong, cmap = 'gray')
plt.title('Edge Tracking by Hysteresis')
plt.xticks([]), plt.yticks([])
```



```
# Plot the original image.
plt.imshow(img, cmap = 'gray')
plt.title('Original Image')
plt.xticks([]), plt.yticks([])
```



```
# Plot the strong pixels.
plt.imshow(strong, cmap = 'gray')
plt.title('Strong Pixels')
plt.xticks([]), plt.yticks([])
```



```
# Plot the weak pixels.
plt.imshow(weak, cmap = 'gray')
plt.title('Weak Pixels
```

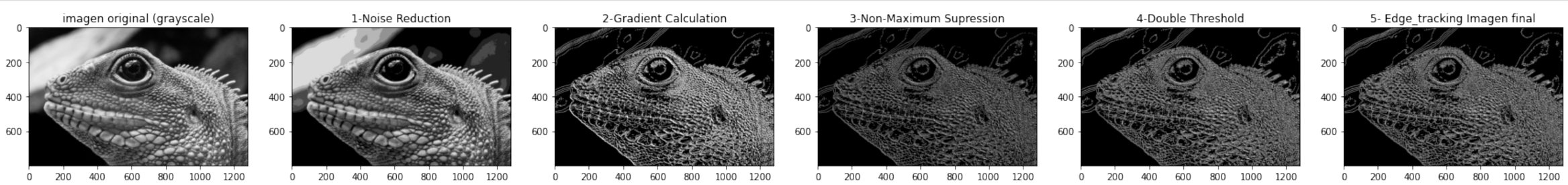
c) Experimenta con imágenes con diferentes cantidad de lineas y textura para observar el comportamiento del algoritmo.

In [27]:
```
#Funcion para ejecutar y plotear todos los pasos del Canny

def canny_steps(img,s):
  plt.figure(figsize=(30, 20))
  img_gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
  plt.subplot(1, 6, 1)
  plt.gca().set_title('imagen original (grayscale)')
  plt.imshow(img_gray, cmap="gray")
  #1. Noise Reduction
  kernel_size = 3
  sigma = s
  img_gaussian = convolve(img_gray, gaussian_kernel(kernel_size, sigma))
  plt.subplot(1, 6, 2)
  plt.gca().set_title('1-Noise Reduction')
  plt.imshow(img_gaussian, cmap="gray")
  #2. Gradient Calculation
  G, theta = sobel_filters(img_gaussian)
  plt.subplot(1, 6, 3)
  plt.gca().set_title('2-Gradient Calculation')
  plt.imshow(G.astype(float), cmap="gray")
  #3. Non-Maximum Supression
  img_nonmax = non_max_suppression(G, theta)
  plt.subplot(1, 6, 4)
  plt.gca().set_title('3-Non-Maximum Supression')
  plt.imshow(img_nonmax, cmap="gray")
  #4. Double Threshold
  img_threshold = threshold(img_nonmax)
  plt.subplot(1, 6, 5)
  plt.gca().set_title('4-Double Threshold')
  plt.imshow(img_threshold, cmap="gray")
  #5. Edge Tracking by Hysteresis
  img_final = hysteresis(img_threshold)
  plt.subplot(1, 6, 6)
  plt.gca().set_title('5- Edge_tracking Imagen final')
  plt.imshow(img_final, cmap="gray")
```

Imagen 1

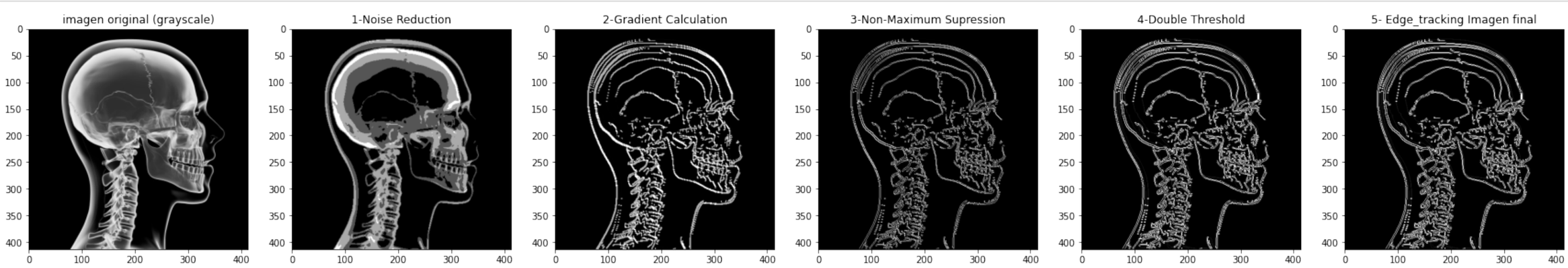
In [28]:
```
img1 = cv2.imread('/content/iguana.jpg')
img_gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
canny_steps(img1,7) # Agregar la imagen y el valor de sigma
```



Conclusión:

En este caso, probamos el detector de bordes de Canny con una imagen de una iguana y un sigma igual a 7. Como podemos observar, en esta imagen se encuentran muchas líneas en todas direcciones y muy cercanas entre ellas debido a las escamas del animal, lo que hace que el detector Canny no elimine gran cantidad de los bordes irrelevantes en la imagen y el resultado se vea muy igual a partir del paso 3 que es el de Non-Maximum Supression. Sin embargo desde el paso 2 al 3 (calculo del grandiente a non-maximum supression) podemos observar que efectivamente los bordes se hacen mucho mas delgados.

In [29]:
```
img2 = cv2.imread('/content/rayos-x.jpg')
img_gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
canny_steps(img2,10) # Agregar la imagen y el valor de sigma
```



Conclusión:

En este caso se aumentó un poco la sigma ya que con valores bajos aumenta la cantidad de líneas que detecta y limita la identificación de objetos en la imagen, como se comento en el ejemplo anterior a partir del paso 3 y 4 reduce el grueso de las líneas y se nota muy parecido el resultado entre el paso 4 y el paso 5.

Referencias:

1. GeeksforGeeks. (2023, 3 enero). Python OpenCV Canny Function. https://www.geeksforgeeks.org/python-opencv-canny-function/
2. Sahir, S. (2021, 7 diciembre). Canny Edge Detection Step by Step in Python — Computer Vision. Medium. https://towardsdatascience.com/canny-edge-detection-step-by-step-in-python-computer-vision-b49c3a2d8123
3. Rosebrock, A. (2021b, mayo 10). OpenCV Edge Detection (cv2.Canny). PyImageSearch. https://pyimagesearch.com/2021/05/12/opencv-edge-detection-cv2-canny/
4. OpenCV: Canny Edge Detector. (s. f.). https://docs.opencv.org/3.4/da/d5c/tutorial_canny_detector.html
5. Rosebrock, A. (2022, 14 abril). Image Gradients with OpenCV (Sobel and Scharr). PyImageSearch. https://pyimagesearch.com/2021/05/12/image-gradients-with-opencv-sobel-and-scharr/